



Rapport de projet Algorithmique Avancée

Écrit par:

AHMED Youra 21416343
BENAISSA Meriem 21418006

Supervisé par les enseignants :

Mme C. Truchet
Mr M. Naima
Mr. Antoine Genitrini

Promotion 2024 - 2025

Contents

1	Introduction	1
2	Structure de donnée	2
2.1	Trie Patricia :	2
2.1.1	Caractère choisi :	2
2.1.2	Strucure choisi :	2
2.1.3	Les primitives de base :	3
2.1.4	Les fonctions avancées :	4
2.1.5	Fonctions complexes :	7
2.2	Hybride Trie :	8
2.2.1	Strucure choisi :	8
2.2.2	Les primitives de base :	8
2.2.3	Les fonctions avancées :	9
2.2.4	Fonctions complexes :	12
3	Étude expérimentale	15
3.1	Temps de construction :	15
3.2	Temps d'insertion d'un nouveau mot :	16
3.3	Temps de recherche :	17
3.4	Temps de suppression :	18
3.5	Comparaison des hauteurs des arbres :	19
3.6	Hauteur et profondeur :	20
3.7	Temps d'insertion avec un trie hybride équilibré :	21
3.8	Hauteurs avec un trie hybride équilibré :	22
4	Conclusion :	23
	Références	24

1 Introduction

Le devoir de programmation proposé dans le cadre de ce projet porte sur l'implémentation et la comparaison de deux structures de données de type trie : les Patricia-Tries et les Tries Hybrides déjà vu en séance de travaux dirigé. Ces structures permettent de représenter un dictionnaire de mots, une tâche courante dans de nombreux domaines informatiques tels que les systèmes de gestion de bases de données, les moteurs de recherche, ou encore les outils de suggestion de texte. L'objectif de ce travail est d'étudier et de mettre en œuvre ces deux structures de manière à pouvoir en analyser les avantages et inconvénients à travers une série d'expérimentations et de mesures de performance.

Nous commencerons par implémenter les primitives de base pour chaque modèle de trie. Ensuite, des fonctions avancées, comme la recherche de mots, le comptage de mots, ou encore la suppression de mots, seront ajoutées à nos structures. Ces fonctions permettront une manipulation complète des arbres, tout en offrant un aperçu sur la manière dont ces structures gèrent des opérations courantes dans un dictionnaire.

Une attention particulière sera portée sur la comparaison expérimentale des deux structures. Différents aspects de performance seront analysés, tels que le temps de construction des arbres, la gestion de l'ajout et de la suppression de mots, ainsi que la profondeur des structures. Cette étude expérimentale permettra de mieux comprendre l'efficacité de chaque modèle dans des situations variées et d'apporter des conclusions sur les scénarios d'utilisation les plus appropriés pour chaque type de trie.

2 Structure de donnée

2.1 Trie Patricia :

Les Patricia-Tries (ou Practical Algorithm to Retrieve Information Coded in Alphanumeric) sont une structure de données dérivée des tries classiques, conçue pour optimiser l'utilisation de la mémoire en minimisant le nombre de nœuds. Elles combinent des chemins ayant des étiquettes communes en un seul nœud, ce qui réduit la profondeur de la hauteur de l'arbre.

2.1.1 Caractère choisi :

Nous avons choisi d'utiliser le caractère Pipe (`|`, ASCII 124) dans ce projet pour les raisons suivantes :

- Il n'est pas utilisé naturellement dans des mots courants ou dans des données linguistiques standards.
- Facile à identifier et à manipuler dans le code source.
- Compatible avec des formats standards et facilement lisible dans des fichiers texte.

Exemple : `"car|cat|cart|dog|bat"`;

2.1.2 Structure choisi :

Notre Patricia-trie est constituée de trois éléments basiques :

- Un label : préfixe partagé.
- `is_end_of_word` : indiquant si le nœud correspond à la fin d'un mot valide.
- Children : Une liste d'enfants stockée sous forme de map, où chaque caractère correspond à un sous-arbre.

2.1.3 Les primitives de base :

- **La fonction *getPrefixeCommun(String s1, String s2)***: qu'a une complexité de $O(n)$ tq n est la longueur de prefixe, cette fonction retourne une chaine de caractère qu'est le prefixe commun entre deux mots.
- **La fonction *arrangement(PatriciaTrieNode parent, PatriciaTrieNode child, String prefixeCommun, String suffix)***: qu'a une complexité de $O(1)$. Elle sert à traiter le cas où le mot à ajouter est un prefixe d'un mots déjà insérer dans la patricia.
- **La fonction *inserer(String mot)*** : qu'a une complexité de $O(n)$ où n est la taille du mot. Elle sert à ajouter un mot dans l'arbre Patricia.
- **La fonction *insertMotduFichier(String filename)*** : qu'a une complexité de $O(n * k)$ où n est le nombre de mots et k est la longueur moyenne des mots. Elle sert à ajouter des mots qui se trouve dans un fichier à un arbre patricia.
- **La fonction *insertMotsDuRepertoire(String repertoirePath)*** : la fonction sert à ajouter des mots qui se trouve dans des fichiers qui sont stocké dans un repertoire.
- ***arbreToJson()*** : Convertit l'arbre en JSON.
- ***saveToFile(String filename)*** : Sauvegarde l'arbre dans un fichier au format JSON.
- ***jsonToArbre(String filename)*** : Charge un arbre depuis un fichier JSON.
- ***getChildren()*** : Accède aux enfants d'un nœud.
- ***setChildren(Map<Character, PatriciaTrieNode> children)*** : Définit les enfants d'un nœud.
- ***setIs_end_of_word(boolean is_end_of_word)*** : Définit si un nœud est la fin d'un mot.
- ***isIs_end_of_word()*** : Vérifie si un nœud est la fin d'un mot.

2.1.4 Les fonctions avancées :

- **La fonction *Recherche*(*PatriciaTrieNode p*, *String mot*):**

La fonction sert à faire la recherche d'un mot dans un arbre patricia.

- Mesure de complexité :

Le nombre de caractères du mot recherché et le nombre

- Complexité :

$O(n)$ où n est la longueur du mot, car on parcourt chaque caractère dans le pire des cas.

- **La fonction *comptageMots*(*PatriciaTrieNode node*):**

La fonction sert à compter le nombre de mots qui existe dans l'arbre Patricia.

- Mesure de complexité :

Le nombre de nœuds dans l'arbre.

- Complexité :

$O(n)$ où n est le nombre total de nœuds dans l'arbre.

- Justification :

La fonction visite tous les nœuds de l'arbre pour vérifier si `is_end_of_word` est vrai et ajoute le mot à la liste. La boucle pour chaque enfant est linéaire par rapport au nombre total de nœuds.

- **La fonction *listeMots*(*PatriciaTrieNode node*):**

La fonction parcourt tous les nœuds et construit chaque mot en accumulant les préfixes et retourne la liste des mots de l'arbre patricia passée en paramètre.

- Mesure de complexité :

Le nombre total de nœuds et le nombre total de mots dans l'arbre.

- Complexité :

$O(n*k*\log(k))$ où n est le nombre de nœuds dans l'arbre, et k est le nombre moyen d'enfants par nœud.

- Justification :

1. Création de TreeMap : Pour chaque nœud, on crée un TreeMap avec une complexité de $O(k \cdot \log(k))$, où k est le nombre d'enfants.
2. Parcours des enfants : Le coût de la boucle for est $O(k)$.
3. Récursion : Pour chaque appel récursif, on fait une nouvelle exploration des enfants.

- **La fonction *comptageNil(PatriciaTrieNode node)*:**

- Mesure de complexité :
Le nombre de nœuds.
- Complexité :
 $O(n)$ où n est le nombre total de nœuds dans l'arbre.
- Justification :
Chaque nœud est visité une fois pour vérifier s'il n'a pas d'enfants.

- **La fonction *hauteur(PatriciaTrieNode node)* :**

La fonction sert à retourner la hauteur de l'arbre patricia passée en paramètre.

- Mesure de complexité :
le nombre de nœuds.
- Complexité :
 $O(n)$ où n est le nombre total de nœuds dans l'arbre.
- Justification :
La fonction effectue un parcours en profondeur. À chaque nœud, elle calcule la hauteur maximale des enfants, ce qui nécessite de parcourir tous les nœuds dans le pire des cas.

- **La fonction *profondeurMoyenne(PatriciaTrieNode node)*:**

La fonction sert à retourner la profondeur moyenne de l'arbre patricia passée en paramètre.

- Mesure de complexité :
Le nombre de nœuds et la profondeur des feuilles.

- Complexité :
O(n) où n est le nombre total de nœuds dans l'arbre.
- Justification :
La fonction visite chaque nœud pour accumuler les profondeurs des feuilles et compter leur nombre.
- **La fonction *prefixe*(*PatriciaTrieNode p*, *String prefix*):**
La fonction sert à retourner la nombre de pe mots dans laquelle le mot passée en paramètre est préfixe.
 - Mesure de complexité :
La longueur du préfixe et le nombre de mots descendants.
 - Complexité :
O(p+k) où p est la longueur du préfixe et k le nombre de mots descendants.
 - Justification :
Parcours des nœuds correspondant au préfixe (O(p), où p est la longueur du préfixe). Une fois le nœud du préfixe trouvé, on compte les mots descendants (O(k), où k est le nombre de mots descendants).
- **La fonction *suppression*(*PatriciaTrieNode node*, *String mot*):**
La fonction sert à supprimer le mot passée en paramètre dans l'arbre patricia.
 - Mesure de complexité :
La longueur du mot et le nombre de nœuds touchés.
 - Complexité :
O(n) où n est la longueur du mot.
 - Justification :
La suppression parcourt les nœuds correspondant au mot (O(n), où n est la longueur du mot). Les opérations de suppression et de réorganisation sont constantes pour chaque nœud touché.

2.1.5 Fonctions complexes :

- **La fonction *fusionner(PatriciaTrieNode trie1, PatriciaTrieNode trie2)*:**

- Mesure de complexité :
Le nombre total de nœuds dans les deux arbres.
- Justification :
La fusion visite tous les nœuds des deux arbres pour les intégrer dans un arbre résultant, avec des opérations proportionnelles au nombre de nœuds.
- Complexité :
 $O(n_1+n_2)$ sont les nombres de nœuds dans les deux arbres.

pour sa réalisation l'idée principale est de fusionner les nœuds des deux arbres en un seul tout en respectant les propriétés des tries, c'est-à-dire en maintenant les préfixes communs et en divisant les nœuds lorsque nécessaire. Voici une explication des étapes importantes de la fusion :

- Initialisation: Créez un nouveau nœud racine (vide ou un nœud de début) pour le résultat final de la fusion.
- Fusion des nœuds : Pour chaque nœud de l'arbre source (*trie1* et *trie2*), vérifiez s'il existe déjà un nœud correspondant dans l'arbre cible (l'arbre de résultat).
- Cas des nœuds existants : Si un nœud avec la même clé (premier caractère) existe dans l'arbre cible, vous devez :
 - * Comparer les préfixes des clés des nœuds source et cible.
 - * Si le préfixe est commun, vous continuez la fusion récursive pour les sous-arbres des nœuds concernés.
 - * Si le préfixe n'est pas commun, vous divisez le nœud en un nouveau nœud commun, puis vous réorganisez les enfants des deux nœuds en fonction du préfixe.
- Cas des nouveaux nœuds : Si un nœud n'existe pas déjà dans l'arbre cible, vous l'ajoutez directement sous le nœud correspondant.

- Répéter pour tous les nœuds : Répétez ce processus pour tous les nœuds dans les arbres trie1 et trie2.

2.2 Hybride Trie :

Le Trie Hybride est une structure de données qui combine les caractéristiques des tries classiques et des arbres binaires de recherche. Chaque nœud représente un caractère et peut avoir trois sous-arbres : gauche, milieu et droit. Cette organisation permet de gérer efficacement les mots en fonction de leur ordre lexicographique. Les tries hybrides offrent un bon compromis entre l'utilisation de la mémoire et la rapidité des opérations, tout en facilitant l'équilibrage de l'arbre pour optimiser les performances, notamment dans le cas de grands dictionnaires.

2.2.1 Structure choisi :

Notre Trie Hybride est constitué de cinq éléments basiques :

- caractere : la valeur du nœud.
- left : Un pointeur vers le fils gauche.
- middle : Un pointeur vers le fils milieu.
- right : Un pointeur vers le fils droit.
- isEndOfWord : Un indicateur de fin de mot.

2.2.2 Les primitives de base :

- **La fonction *insert(TrieHybridesNode trieH, String mot)*** : qu'a une complexité de $O(n)$ où n est la taille du mot. Elle sert à ajouter un mot dans l'arbre Hybride.
- **La fonction *insertMotduFichier(String filename)*** : qu'a une complexité de $O(n * k)$ où n est le nombre de mots et k est la longueur moyenne des mots. Elle sert à ajouter des mots qui se trouve dans un fichier à un arbre Hybride.
- **La fonction *insertMotsDuRepertoire(String repertoirePath)*** : la fonction sert à ajouter des mots qui se trouve dans des fichiers qui sont stocké dans un répertoire.

- **La fonction *arbreToJson()*** : Fonction qui convertit l'arbre hybride en JSON.
- **La fonction *saveToFile(String filename)*** : Fonction qui sauvegarde l'arbre hybride dans un fichier au format JSON.
- **La fonction *jsonToArbre(String filename)*** : Fonction qui charge un arbre hybride depuis un fichier JSON.
- **La fonction *rotationDroite(TrieHybridesNode arbre)*** : Fonction qui effectue une rotation droite sur un arbre hybride.
- **La fonction *rotationGauche(TrieHybridesNode arbre)*** : Fonction qui effectue une rotation gauche sur un arbre hybride.

2.2.3 Les fonctions avancées :

- **La fonction *Recherche(TrieHybridesNode trieH, String mot)***:
La fonction sert à faire la recherche d'un mot dans un arbre hybride.
 - Mesure de complexité :
Le nombre de caractères du mot recherché et le nombre de nœuds.
 - Complexité :
 $O(n + h)$ où n est la longueur du mot et h est la hauteur de l'arbre.
- **La fonction *comptageMots(TrieHybridesNode node)***:
La fonction sert à compter le nombre de mots qui existe dans l'arbre Hybride.
 - Mesure de complexité :
Le nombre de nœuds dans l'arbre.
 - Complexité :
 $O(n)$ où n est le nombre total de nœuds dans l'arbre.
 - Justification :
La fonction visite tous les nœuds de l'arbre pour vérifier si `sEndOfWord` est vrai et ajoute le mot à la liste. La boucle pour chaque enfant est linéaire par rapport au nombre total de nœuds.

- **La fonction *listeMots(TrieHybridesNode trieHY)*:** Cette fonction sert à lister tous les mots stockés dans le trie hybride par ordre alphabétique.
 - Mesure de complexité : Le nombre total de nœuds dans le trie hybride.
 - Complexité :
 $O(n)$, où n est le nombre total de nœuds dans le trie hybride.
 - Justification :
La fonction parcourt récursivement tous les nœuds du trie hybride pour vérifier s'ils marquent la fin d'un mot et les ajouter à la liste. Chaque nœud est visité une seule fois, et les opérations effectuées par nœud (vérifications et concaténations de chaînes) sont de complexité constante $O(1)$. Par conséquent, la complexité totale est linéaire par rapport au nombre total de nœuds dans le trie.
- **La fonction *comptageNil(TrieHybridesNode trieHY)*:** Cette fonction compte le nombre de pointeurs null (Nil) dans le trie hybride.
 - Mesure de complexité :
Le nombre total de nœuds dans le trie hybride.
 - Complexité :
 $O(n)$, où n est le nombre total de nœuds dans le trie hybride.
 - Justification :
Chaque nœud est visité exactement une fois, et pour chaque nœud, la fonction effectue un appel récursif sur ses trois pointeurs. Ainsi, la complexité est linéaire par rapport au nombre total de nœuds.
- **La fonction *hauteur(TrieHybridesNode trieHY)*:**
La fonction sert à retourner la hauteur de l'arbre hybride passée en paramètre.
 - Mesure de complexité :
Le nombre de nœuds dans l'arbre hybride.
 - Complexité :
 $O(n)$ où n est le nombre total de nœuds dans l'arbre hybride.

- Justification :
La fonction effectue un parcours en profondeur. À chaque nœud, elle calcule la hauteur maximale des enfants, ce qui nécessite de parcourir tous les nœuds dans le pire des cas.
- **La fonction *profondeurMoyenne(TrieHybridesNode trieHY)*:**
La fonction sert à retourner la profondeur moyenne de l'arbre hybride passée en paramètre.
 - Mesure de complexité :
Le nombre de nœuds et la profondeur des feuilles.
 - Complexité :
 $O(n)$ où n est le nombre total de nœuds dans l'arbre hybride.
 - Justification :
Chaque nœud est visité une seule fois lors du parcours en profondeur.
- **La fonction *prefixe(TrieHybridesNode trieH, String prefix)*:**
Cette fonction sert à retourner le nombre de mots dans lesquels le mot passé en paramètre est un préfixe.
 - Mesure de complexité :
La longueur du préfixe, notée m , et le nombre de nœuds du sous-trie correspondant.
 - Complexité :
 $O(m + k)$, où m est la longueur du préfixe et k est le nombre de nœuds du sous-trie correspondant.
 - Justification :
La fonction parcourt le préfixe pour trouver le sous-trie correspondant, ce qui prend $O(m)$. Ensuite, elle compte les mots dans ce sous-trie, ce qui prend $O(k)$. La complexité totale est donc $O(m + k)$.
- **La fonction *suppression(TrieHybridesNode trieH, String mot)*:**
La fonction sert à supprimer le mot passé en paramètre dans l'arbre hybride.

- **Mesure de complexité :**
La hauteur du trie, notée h , et la longueur du mot, notée m .
- **Complexité :**
 $O(h + m)$, où h est la hauteur du trie et m est la longueur du mot à supprimer.
- **Justification :**
La fonction parcourt le mot caractère par caractère, ce qui a une complexité de $O(m)$. Cependant, la suppression peut nécessiter de remonter l'arbre jusqu'à la racine pour ajuster les pointeurs lorsque les nœuds deviennent inutiles. Cette remontée dépend de la hauteur du trie, ce qui ajoute une complexité de $O(h)$. Ainsi, la complexité totale est $O(h + m)$.

2.2.4 Fonctions complexes :

Rééquilibrage dans le Trie Hybride

L'un des aspects essentiels de la gestion du Trie Hybride est le rééquilibrage automatique des nœuds pour maintenir des performances optimales. Voici une explication détaillée des fonctions complexes associées à cette opération :

- **La fonction *rotationGauche(TrieHybridesNode arbre)*:**

- **Mesure de complexité :** Le nombre total de nœuds dans le sous-arbre affecté.
- **Complexité :** $O(1)$ (opération constante)
- **Justification :** La rotation gauche modifie uniquement les pointeurs de deux nœuds, ce qui prend un temps constant. Aucun parcours supplémentaire n'est nécessaire.
- **Description :** Cette fonction effectue une rotation gauche sur le nœud spécifié pour maintenir l'équilibre. Voici les étapes principales :
 1. Vérification si l'arbre ou son sous-arbre droit est null. Si c'est le cas, l'arbre reste inchangé.
 2. L'enfant droit devient la nouvelle racine temporaire.
 3. L'ancien nœud racine devient l'enfant gauche du nouveau nœud racine.

4. Mise à jour des pointeurs pour refléter la rotation.

- **La fonction *rotationDroite(TrieHybridesNode arbre)*:**

- **Mesure de complexité :** Le nombre total de nœuds dans le sous-arbre affecté.
- **Complexité :** $O(1)$ (opération constante)
- **Justification :** Comme pour la rotation gauche, la rotation droite ajuste simplement les pointeurs, ce qui se fait en temps constant.
- **Description :** Cette fonction effectue une rotation droite pour équilibrer l'arbre hybride :
 1. Vérification si l'arbre ou son sous-arbre gauche est null.
 2. L'enfant gauche devient la nouvelle racine temporaire.
 3. L'ancien nœud racine devient l'enfant droit du nouveau nœud racine.
 4. Mise à jour des pointeurs.

- **La fonction *equilibrer(TrieHybridesNode arbre)*:**

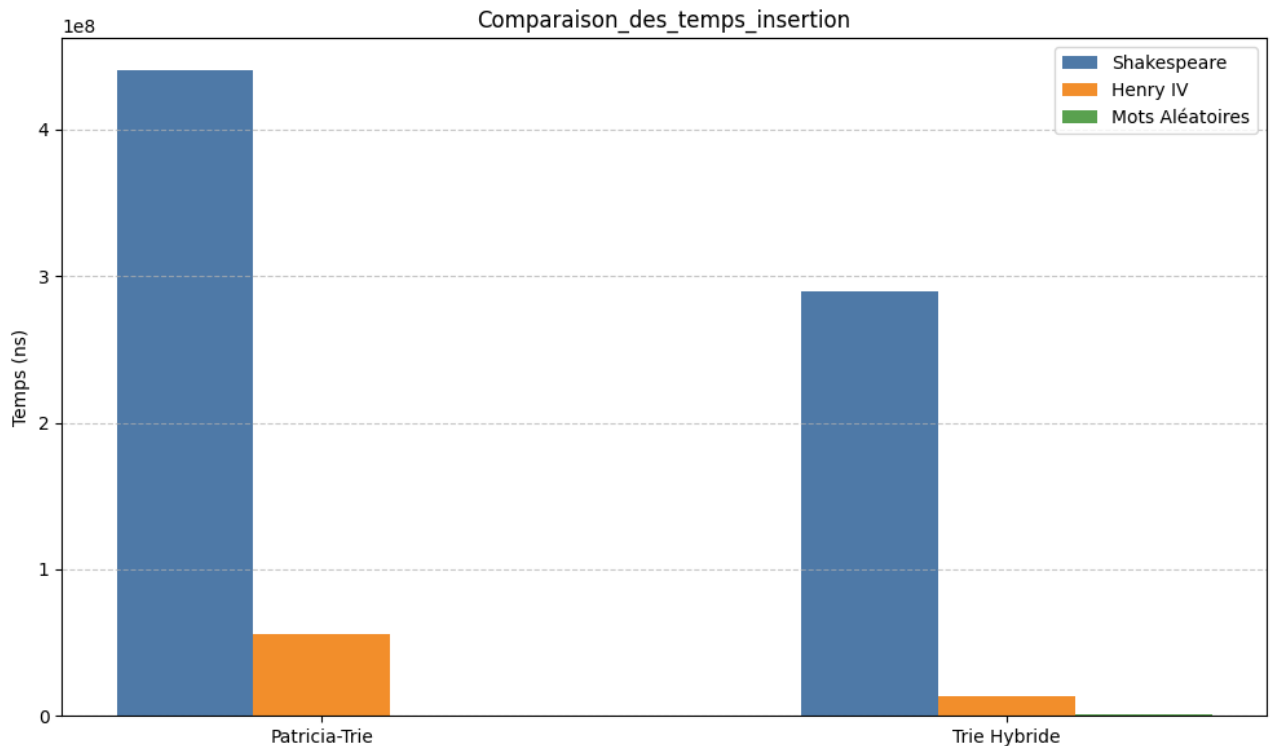
- **Mesure de complexité :** La hauteur de l'arbre.
- **Complexité :** $O(h)$ où h est la hauteur du trie hybride.
- **Justification :** Cette fonction vérifie la hauteur des sous-arbres gauche et droit du nœud actuel. Si l'écart dépasse un seuil (ici 2), une rotation simple ou double est effectuée pour rééquilibrer l'arbre.
- **Description :** Le processus de rééquilibrage est le suivant :
 1. Calculer les hauteurs des sous-arbres gauche et droit.
 2. Si la différence de hauteur est supérieure ou égale à 2 :
 - * Si le sous-arbre gauche est plus profond :
 - Vérifier si une rotation gauche doit être effectuée sur le sous-arbre gauche.
 - Effectuer une rotation droite sur le nœud courant.
 - * Si le sous-arbre droit est plus profond :

- Vérifier si une rotation droite doit être effectuée sur le sous-arbre droit.
 - Effectuer une rotation gauche sur le nœud courant.
 - 3. Retourner le nœud rééquilibré.
- **La fonction *insertEtReequilibrer(TrieHybridesNode arbre, String mot)*:**
 - **Mesure de complexité :** La longueur du mot et la hauteur de l'arbre.
 - **Complexité :** $O(m + h)$ où m est la longueur du mot et h est la hauteur du trie hybride.
 - **Justification :** L'insertion a une complexité de $O(m)$, car chaque caractère du mot est traité. Le rééquilibrage ajoute un coût supplémentaire de $O(h)$ dans le pire des cas.
 - **Description :** Cette fonction combine l'insertion et le rééquilibrage de l'arbre :
 1. Vérification si le mot est nul ou vide.
 2. Insertion du mot dans l'arbre hybride.
 3. Appel à la fonction `equilibrer` pour rééquilibrer l'arbre après l'insertion.

3 Étude expérimentale

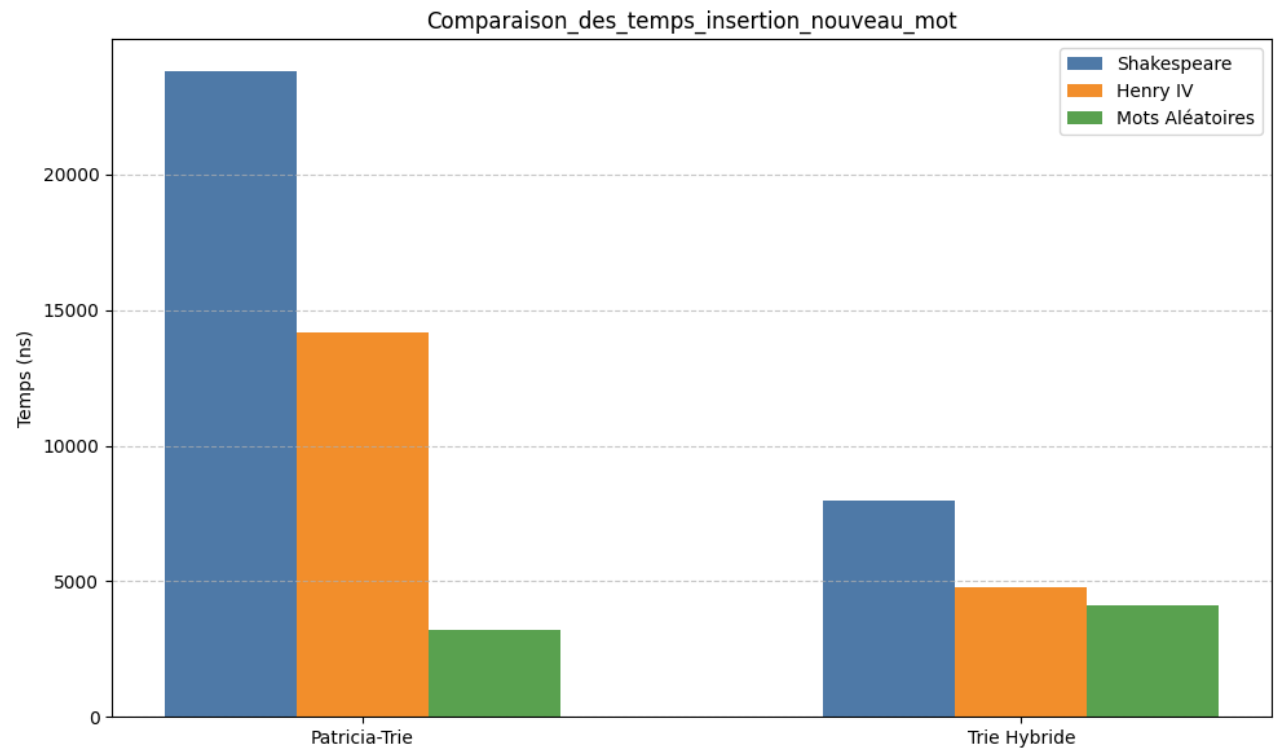
Après avoir effectué quelques tests sur les arbres avec différentes entrées (mots aléatoires, l'un des fichiers Shakespeare et tout l'oeuvre de Shakespeare), on a eu les résultats suivants :

3.1 Temps de construction :



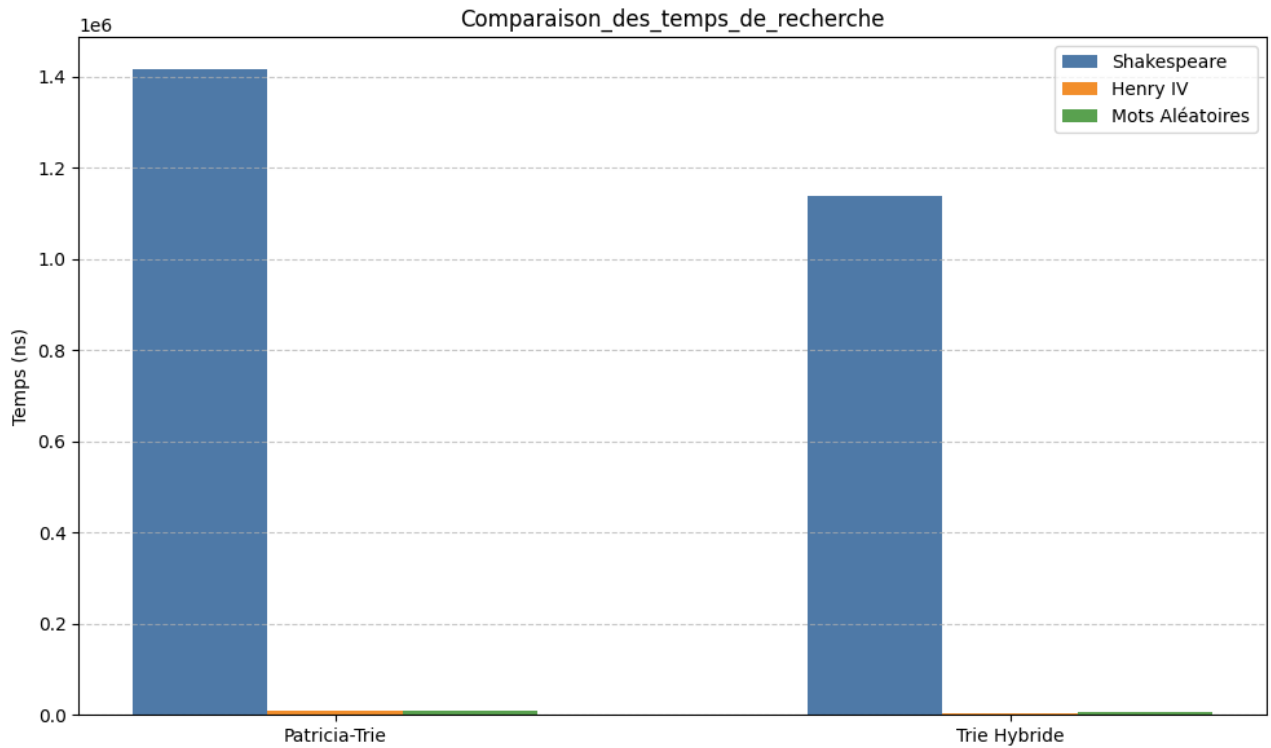
Ce diagramme montre clairement que la construction se fait plus rapidement dans les tries hybrides, de ce fait le trie hybride est meilleur en terme de rapidité.

3.2 Temps d'insertion d'un nouveau mot :



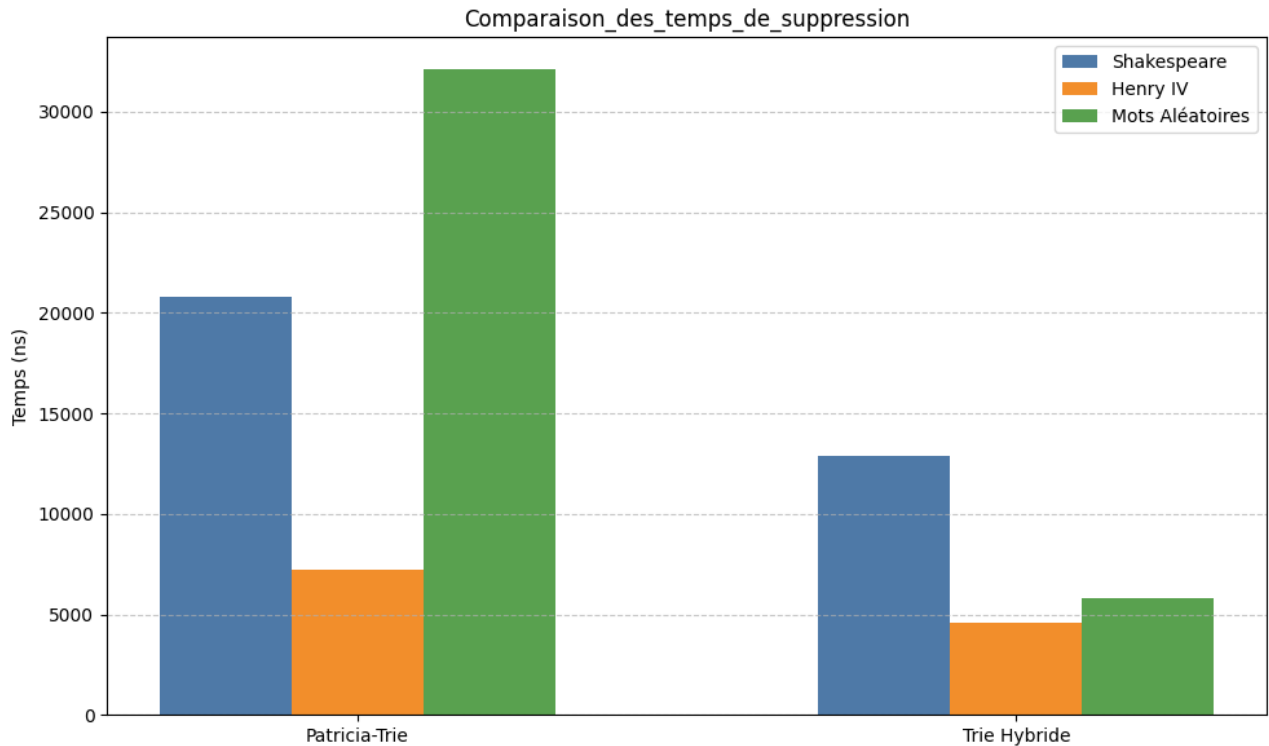
Ce diagramme montre aussi que quand on insere un nouveau mot dans un arbre, le trie hybride est plus rapide que le patricia trie.

3.3 Temps de recherche :



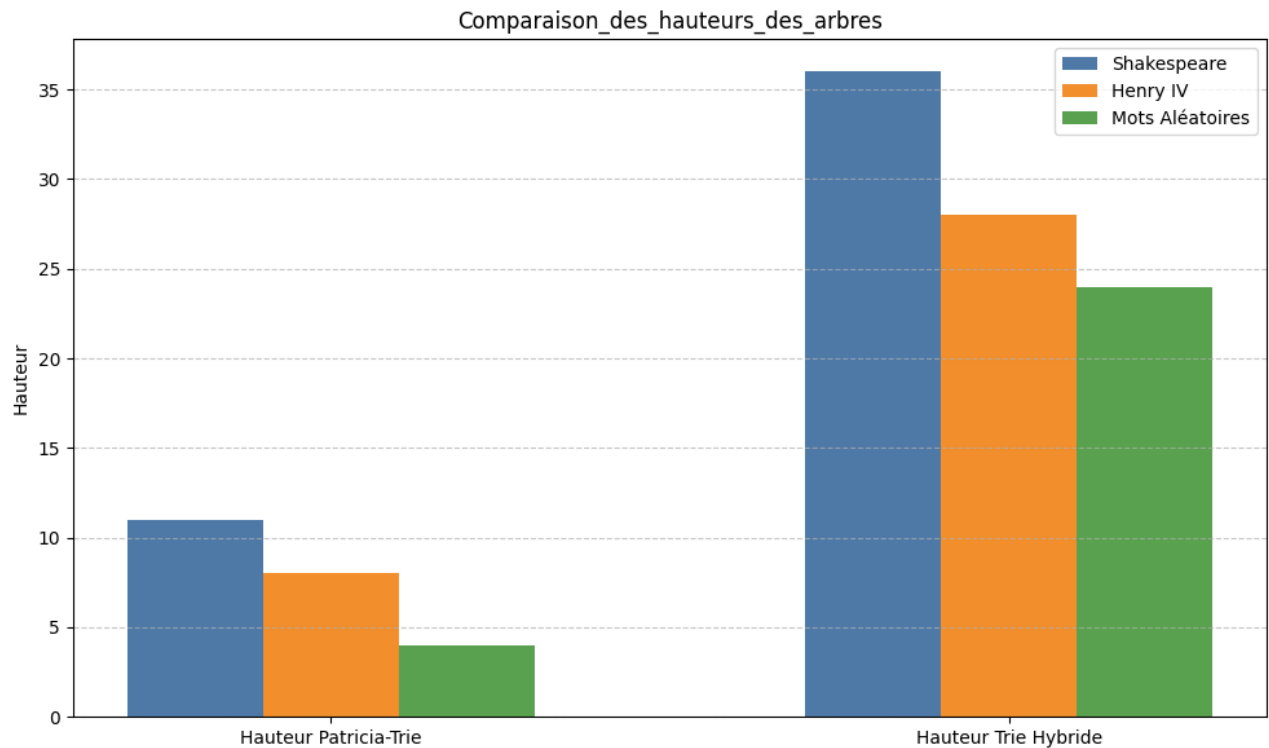
Ce diagramme montre que le trie hybride est plus rapide que le patricia trie en temps de recherche.

3.4 Temps de suppression :



Sur ce diagramme, on peut voir que le trie hybride est plus rapide que le patricia trie en temps de suppression.

3.5 Comparaison des hauteurs des arbres :



Dans ce diagramme, on peut voir clairement que le trie hybride a une hauteur plus élevée que le patricia trie, cela est dû au fait que la trie hybride crée plus de nœuds que le patricia trie.

3.6 Hauteur et profondeur

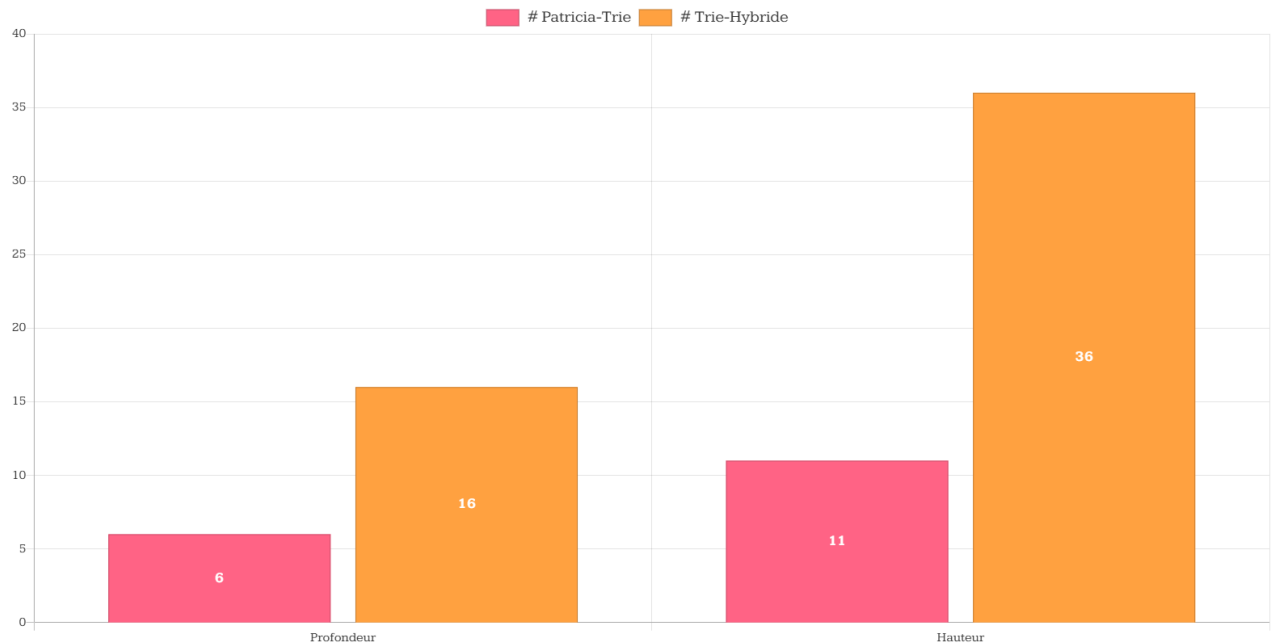


diagramme pour la profondeur et la hauteur des arbres

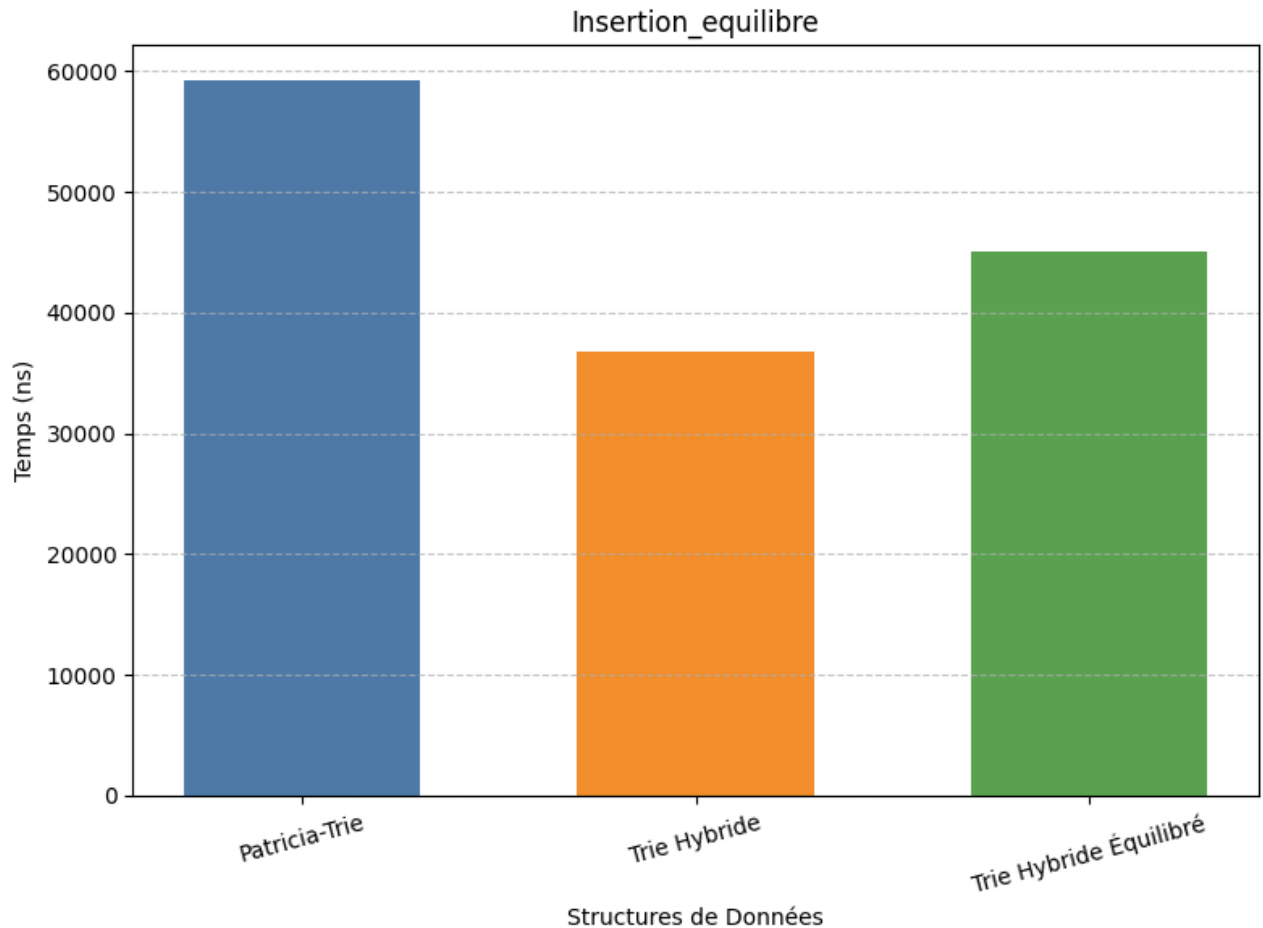
ce diagramme montre que la hauteur et la profondeur de l'arbre stockant l'oeuvre de shakespeare est plus élevé dans la Trie-Hybride que dans la Patricia-Trie, donc on a pensé à mesurer la taille des fichiers json qui stock l'oeuvre de shakespeare et ça à donner les résultats suivant :

Patricia-Trie: 5442035 octets.

Trie Hybride: 16868237 octets.

d'après cette experimentation on a conclu que les patricia-tries est meilleur en terme d'occupation d'espace mémoire.

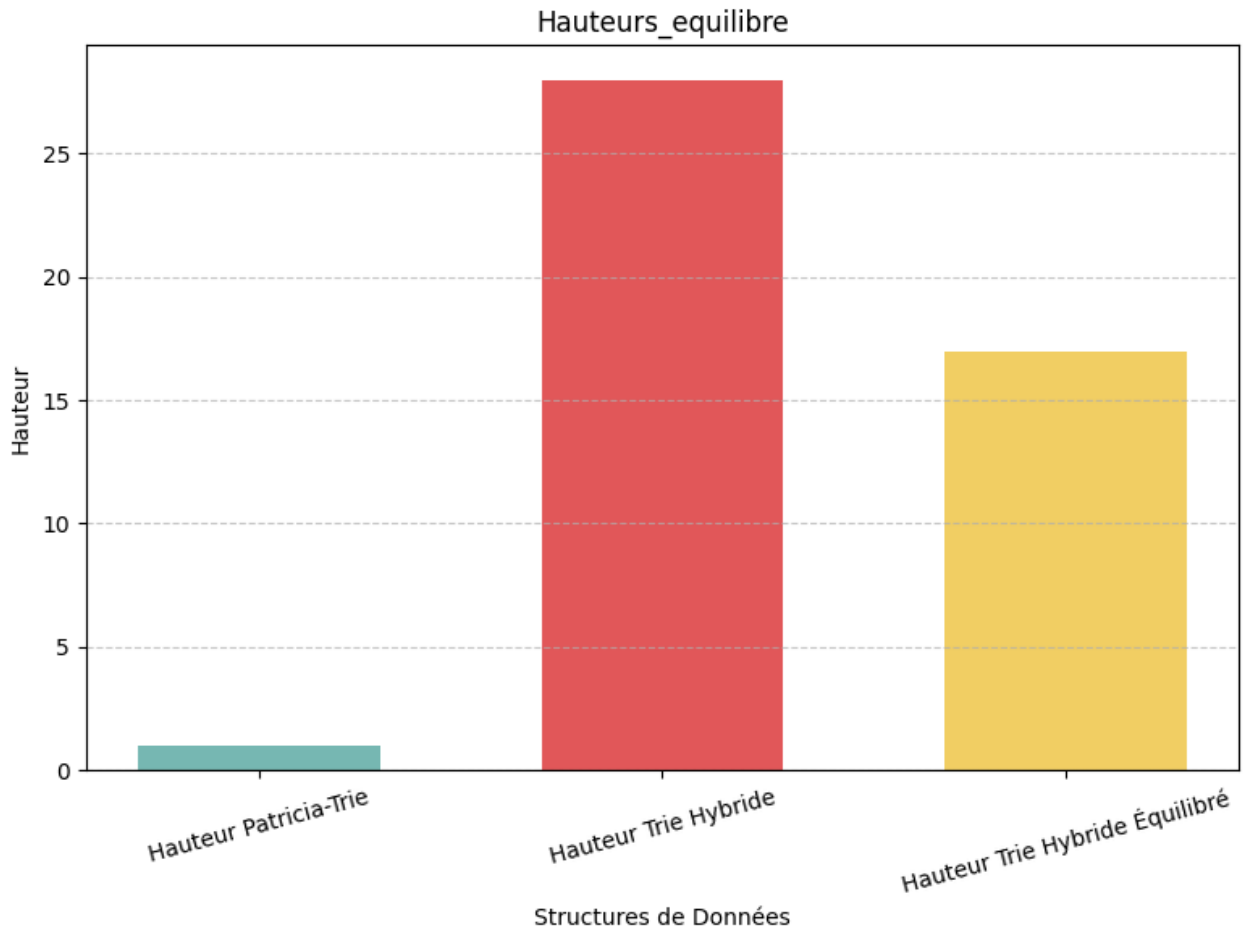
3.7 Temps d'insertion avec un trie hybride équilibré :



Ici on peut voir que le trie hybride est plus rapide que le trie hybride équilibré en terme d'insertion car le trie hybride équilibré doit faire beaucoup de comparaisons pour insérer un mot. Et la patricia trie est toujours lente en terme d'insertion.

Remarque : le graphe est un résultat d'une experimentation sur des arbres qu'ont des mots qu'on a rajouter nous même.

3.8 Hauteurs avec un trie hybride équilibré :



Dans ce diagramme, on peut voir que le trie hybride équilibré a une hauteur plus faible que le trie hybride non équilibré, mais on peut constater aussi que la hauteur de la patricia trie est plus faible que celle du trie hybride équilibré.

4 Conclusion :

pour les deux type d'arbre qu'on a vu, chacune a ses avantages et inconvénients, la Patricia trie est meilleur en terme d'occupation d'espace mémoire, et moins bonne en terme de rapidité. Pour la trie hibryde, elle comme avantage sa rapidité d'insertion, recherche, suppression en terme de temps et a comme inconvénient l'occupation de l'espace mémoire.

References

- [1] Eclipse IDE :<https://www.eclipse.org/downloads/>
- [2] Overleaf pour le rapport :<https://www.overleaf.com/project>
- [3] JDK JRE : <https://www.oracle.com/java/technologies/downloads/>
- [3] Python : <https://www.python.org/downloads/>